CS 598 PSL Fall 2026

Coding Assignment 3

Due Date: Monday, October 12 (11.59pm)

- Anirudh Eswara
- Ryan Lenea
- Mark Synowiec
- Robert Walker

This coding assignment uses the CollegeScores4yr table in the R package Lock5Data (U.S. Department of Education College Scorecard). Each row is a U.S. institution that primarily grants bachelor's degrees. The response is CompRate: the share of students who finish a program within 150% of normal time.

Predictors are institutional attributes such as public / private / for-profit Control, Locale, Region, admission rate, test scores, enrollment, student demographics, net price, faculty salary, instructional spending, and Pell share.

The main task is to use tree-based methods to predict completion (CompRate), compare them to simple baselines and one linear model, and then write a short discussion on eight test-set schools.

In [1]:
```
import pandas as pd
import numpy as np

file_path = "CollegeScores4yr.csv"

try:
    df = pd.read_csv(file_path)

    display(df.head())
except Exception as e:
    print(f"An unexpected error occurred: {e}")
```

	Name	State	ID	Main	Accred	MainDegree	HighDegree	Control	Regic
0	Alabama A & M University	AL	100654	1	Southern Association of Colleges and Schools C...	3	4	Public	Southea
1	University of Alabama at Birmingham	AL	100663	1	Southern Association of Colleges and Schools C...	3	4	Public	Southea
2	Amridge University	AL	100690	1	Southern Association of Colleges and Schools C...	3	4	Private	Southea
3	University of Alabama in Huntsville	AL	100706	1	Southern Association of Colleges and Schools C...	3	4	Public	Southea
4	Alabama State University	AL	100724	1	Southern Association of Colleges and Schools C...	3	4	Public	Southea

5 rows × 35 columns

The data is quite messy, so please take the following steps before splitting into training/testing sets:

- Drop rows with missing CompRate.
- Drop Accred (too many agencies) and State (use Region for geography).
- Drop MainDegree. It is 3 for every school in CollegeScores4yr, so it is not a predictor. Leaving it in will cause lm to fail.
- Keep Name and ID attached to the same rows when you split, but do not put them in the model.
- Add missingness indicators for AdmitRate, MidACT, and AvgSAT (1 if that field is missing, 0 otherwise). Test scores and admit rates are not missing at random.
- Latitude and longitude are optional; they are not needed for a good model.

In [2]:
```
print(df.columns)
```

```
Index(['Name', 'State', 'ID', 'Main', 'Accred', 'MainDegree', 'HighDegree',
       'Control', 'Region', 'Locale', 'Latitude', 'Longitude', 'AdmitRate',
       'MidACT', 'AvgSAT', 'Online', 'Enrollment', 'White', 'Black',
       'Hispanic', 'Asian', 'Other', 'PartTime', 'NetPrice', 'Cost',
       'TuitionIn', 'TuitionOut', 'TuitionFTE', 'InstructFTE', 'FacSalary',
       'FullTimeFac', 'Pell', 'CompRate', 'Debt', 'PctWomen'],
      dtype='str')
```

In [3]:
```python
# cleanup dataset
cleaned_df = df.dropna(subset="CompRate")
cleaned_df = cleaned_df.drop(columns=["Accred", "State", "MainDegree"])

cleaned_df["AdmitRateMissing"] = cleaned_df["AdmitRate"].isna()
cleaned_df["MidACTMissing"] = cleaned_df["MidACT"].isna()
cleaned_df["AvgSATMissing"] = cleaned_df["AvgSAT"].isna()

#optional columns
cleaned_df = cleaned_df.drop(columns=["Latitude", "Longitude"])

y = cleaned_df["CompRate"]
X = cleaned_df.drop(columns=["CompRate"])

print(X.columns)
```

```
Index(['Name', 'ID', 'Main', 'HighDegree', 'Control', 'Region', 'Locale',
       'AdmitRate', 'MidACT', 'AvgSAT', 'Online', 'Enrollment', 'White',
       'Black', 'Hispanic', 'Asian', 'Other', 'PartTime', 'NetPrice', 'Cost',
       'TuitionIn', 'TuitionOut', 'TuitionFTE', 'InstructFTE', 'FacSalary',
       'FullTimeFac', 'Pell', 'Debt', 'PctWomen', 'AdmitRateMissing',
       'MidACTMissing', 'AvgSATMissing'],
      dtype='str')
```

Part I: Tree-based methods

(a) Split the data into training (70%) and testing (30%) data sets. Use seed 598 for reproducibility of the results. Keep Name and ID aligned with the same rows.

After the split, and after the training-only fill-in, compute two baselines on the test set:

(i) the training-mean of CompRate for every test row;

(ii) the training-mean of CompRate within each Control level. Report both MSEs.

After the split (training statistics only):

- For every remaining numeric predictor, fill missing values with the training median. Apply those same medians to the test set.
- For every remaining categorical predictor, fill missing values with the training mode. If the test set contains a factor level that never appears in training, map it to that mode.
- Do not drop additional rows, and do not compute fill-in values from the test set or from the combined data.

In [4]:

```
import sklearn
from sklearn.metrics import mean_squared_error
from sklearn.preprocessing import OneHotEncoder

RANDOM_SEED = 598

def OneHotColumns(df, cols, enc=None):
    # sparse_output=False so transform returns a dense array pd.DataFrame can
consume directly
    if enc is None:
        enc = OneHotEncoder(handle_unknown="ignore", sparse_output=False)
        enc.fit(df[cols])
    encoded = enc.transform(df[cols])
    features = enc.get_feature_names_out(cols)
    print(encoded.shape)
    new_df = pd.DataFrame(encoded, columns=features, index=df.index)
    return pd.concat([new_df, df.drop(columns=cols)], axis=1), enc

# split into training and testing sets
X_train, X_test, y_train, y_test = sklearn.model_selection.train_test_split(X, y,
test_size=0.3, random_state=RANDOM_SEED)

# fill in missing values (training statics from above)
num_cols = X_train.select_dtypes(include=np.number).columns
num_means = X_train[num_cols].mean()
X_train[num_cols] = X_train[num_cols].fillna(num_means)
X_test[num_cols] = X_test[num_cols].fillna(num_means)

cat_cols = X_train.select_dtypes(include="string").columns
cat_modes = X_train[cat_cols].mode()
X_train[cat_cols] = X_train[cat_cols].fillna(cat_modes)
X_test[cat_cols] = X_test[cat_cols].fillna(cat_modes)

X_train_nameid = X_train[["Name", "ID"]]
X_test_nameid = X_test[["Name", "ID"]]

X_train = X_train.drop(columns=["Name", "ID"])
X_test = X_test.drop(columns=["Name", "ID"])

# capture Control before one-hot encoding replaces it with Control_* dummy columns
control_train = X_train["Control"]
control_test = X_test["Control"]

X_train, cat_encoder = OneHotColumns(X_train, cat_cols.drop("Name"))
X_test, _ = OneHotColumns(X_test, cat_cols.drop("Name"), enc=cat_encoder)

# compute two baselines:
# training mean of CompRate for every test row, and
# training mean of CompRate within each Control level
mean = y_train.mean()
mean_private = y_train[control_train == "Private"].mean()
mean_public = y_train[control_train == "Public"].mean()
mean_profit = y_train[control_train == "Profit"].mean()
```

```
# report both MSEs
print("(i) Overall MSE")
print(mean_squared_error(y_test, np.full((len(y_test)), mean)))


print("(ii) Private MSE")
y_test_private = y_test[control_test == "Private"]
print(mean_squared_error(y_test_private, np.full((len(y_test_private)),
mean_private)))

print("(ii) Public MSE")
y_test_public = y_test[control_test == "Public"]
print(mean_squared_error(y_test_public, np.full((len(y_test_public)),
mean_public)))

print("(ii) Profit MSE")
y_test_profit = y_test[control_test == "Profit"]
print(mean_squared_error(y_test_profit, np.full((len(y_test_profit)),
mean_profit)))

#print(cr_mean, cr_profit_mean, cr_public_mean, cr_profit_mean)
```

```
(1278, 12)
(548, 12)
(i) Overall MSE
405.409540640721
(ii) Private MSE
398.70959931510845
(ii) Public MSE
336.93469492614
(ii) Profit MSE
655.5298937073596
```

In [5]:
```
display(X_train.columns)
display(cat_cols.drop("Name"))
```

```
Index(['Control_Private', 'Control_Profit', 'Control_Public', 'Region_Midwest',
       'Region_Northeast', 'Region_Southeast', 'Region_Territories',
       'Region_West', 'Locale_City', 'Locale_Rural', 'Locale_Suburb',
       'Locale_Town', 'Main', 'HighDegree', 'AdmitRate', 'MidACT', 'AvgSAT',
       'Online', 'Enrollment', 'White', 'Black', 'Hispanic', 'Asian', 'Other',
       'PartTime', 'NetPrice', 'Cost', 'TuitionIn', 'TuitionOut', 'TuitionFTE',
       'InstructFTE', 'FacSalary', 'FullTimeFac', 'Pell', 'Debt', 'PctWomen',
       'AdmitRateMissing', 'MidACTMissing', 'AvgSATMissing'],
      dtype='str')
Index(['Control', 'Region', 'Locale'], dtype='str')
```

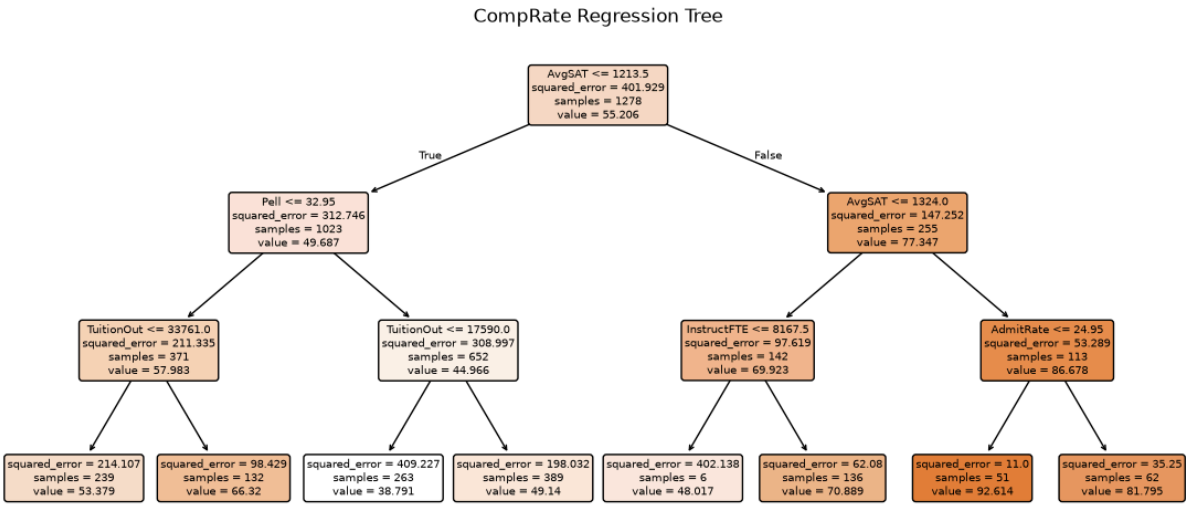
(b) Fit a tree model and visualize it. Interpret at least two splits. Say whether each split looks more like student mix (Pell, demographics), selectivity (admit rate, SAT/ACT), or resources (faculty salary, instructional spending).

In [6]:
```
import matplotlib.pyplot as plt

from sklearn.model_selection import GridSearchCV, train_test_split
from sklearn.tree import DecisionTreeRegressor, plot_tree
```

```
tree = DecisionTreeRegressor(
    max_depth=3,
    min_impurity_decrease=0.008,
    random_state=RANDOM_SEED,
)
tree.fit(X_train, y_train)

plt.figure(figsize=(14, 6))
plot_tree(
    tree,
    feature_names=list(X_train.columns),
    filled=True,
    rounded=True,
    fontsize=7,
)
plt.title("CompRate Regression Tree")
plt.show()
```



The top split was based on selectivity (Average SAT), with higher selectivity correlating with a higher completion rate (77.3%, compared to 49.7%). For less selective schools, completion rate was next split on student data (Pell, the share of students receiving Pell grants), where schools with a lower Pell share were more likely to have higher completion (58.0%) than schools with a higher Pell share (45.0%). For highly selective schools, we see the second split on the right continue to split on Average SAT, continuing the trend that the more selective the school, the higher the completion rate (86.7%, compared to 69.9%). For the left half of the tree (less selective schools), interestingly, the final level of splits was based on Out of State Tuition (Resources). Different features were used to split at the third layer on the right side of the tree. Selectivity (Admission Rate) split the right most leaf nodes, with the lowest admission rate group at the far right. The final split, for schools with Average SAT score greater than 1213.5 and less than 1324.0, was Instructional Spending per FTE Student (Resources).

(c) Prune the tree using cross-validation. Compare test MSE of the pruned vs. unpruned tree.Then grow a second tree using only the public training rows. Do the first two splits match the pooled tree? If not, the pooled tree is averaging different kinds of institution together. Comment on what changed.

In [7]:
```
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import mean_squared_error
from sklearn.tree import DecisionTreeRegressor, plot_tree, export_text

#  baseline tree
unpruned_tree = DecisionTreeRegressor(random_state=RANDOM_SEED).fit(X_train,
y_train)
unpruned_mse = mean_squared_error(y_test, unpruned_tree.predict(X_test))

# prune
alphas = unpruned_tree.cost_complexity_pruning_path(X_train, y_train).ccp_alphas
grid = GridSearchCV(
    DecisionTreeRegressor(random_state=RANDOM_SEED),
    {"ccp_alpha": alphas},
    cv=5,
    scoring="neg_mean_squared_error",
)
grid.fit(X_train, y_train)
pruned_tree = grid.best_estimator_
pruned_mse = mean_squared_error(y_test, pruned_tree.predict(X_test))

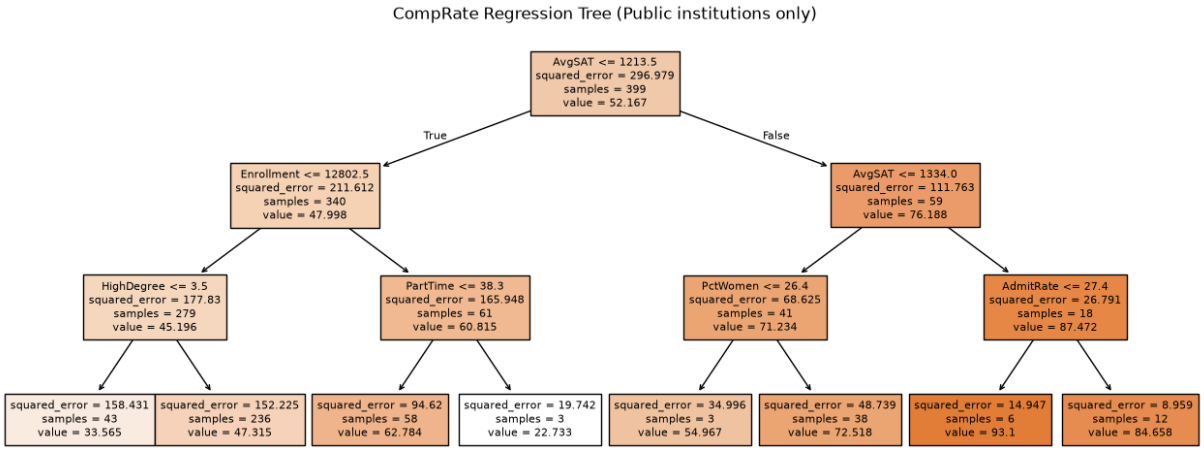
print("Unpruned test MSE:", unpruned_mse)
print("Pruned test MSE:", pruned_mse, "| ccp_alpha:",
grid.best_params_["ccp_alpha"])

public_tree = DecisionTreeRegressor(max_depth=3, min_impurity_decrease=0.008,
random_state=RANDOM_SEED)
public_tree.fit(X_train[control_train == "Public"], y_train[control_train ==
"Public"])

plt.figure(figsize=(16, 6))
plot_tree(public_tree, feature_names=list(X_train.columns), filled=True,
fontsize=8)
plt.title("CompRate Regression Tree (Public institutions only)")
plt.show()

print("Pooled tree (from part b), top splits:")
print(export_text(tree, feature_names=list(X_train.columns), max_depth=1))
print("Public-only tree, top splits:")
print(export_text(public_tree, feature_names=list(X_train.columns), max_depth=1))
```

```
Unpruned test MSE: 327.0156386861314
Pruned test MSE: 210.12377668826724 | ccp_alpha: 5.332026816564323
```




```
Pooled tree (from part b), top splits:
|--- AvgSAT <= 1213.50
|   |--- Pell <= 32.95
|   |   |--- truncated branch of depth 2
|   |--- Pell >  32.95
|   |   |--- truncated branch of depth 2
|--- AvgSAT >  1213.50
|   |--- AvgSAT <= 1324.00
|   |   |--- truncated branch of depth 2
|   |--- AvgSAT >  1324.00
|   |   |--- truncated branch of depth 2

Public-only tree, top splits:
|--- AvgSAT <= 1213.50
|   |--- Enrollment <= 12802.50
|   |   |--- truncated branch of depth 2
|   |--- Enrollment >  12802.50
|   |   |--- truncated branch of depth 2
|--- AvgSAT >  1213.50
|   |--- AvgSAT <= 1334.00
|   |   |--- truncated branch of depth 2
|   |--- AvgSAT >  1334.00
|   |   |--- truncated branch of depth 2
```

The pruned tree did better: its test MSE was 210, compared with 327 for the unpruned tree. Pruning with cross-validation removed splits that were fitting noise in the training data, so the smaller tree generalized better. Both the pooled and public-only trees start with AvgSAT at the same cutoff (1213), so the first split matches. The second splits do not fully match. On the left (lower SAT), the pooled tree uses Pell (student mix, at 32.95), while the public-only tree uses Enrollment (at 12802.5). Among less selective public schools, the larger schools (Enrollment above 12802.5) have a higher average completion rate (60.8%) than the smaller ones (45.2%). On the right (higher SAT), both trees split on AvgSAT again, but at slightly different cutoffs (1324.0 for pooled, 1334.0 for public-only). The third layer of the public-only tree uses HighDegree and PartTime on the left, and PctWomen and AdmitRate on the right. None of these appeared in the pooled tree. This suggests the pooled tree is averaging different kinds of institutions together. Pell stands in for the public/private/profit mix

in the pooled tree, but once we only look at public schools, school size and the type of student body (part-time share, degree level) matter more. Selectivity (AvgSAT) is the one thing that stays important in both trees.

(d) Fit a random forest model to the training data. Choose two different values for mtry (in R) or max features (in Python). If you use R, set importance=TRUE. In Python this is automatic with RandomForestRegressor. Comment on the results.

In [8]:
```
import pandas as pd
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error

n_features = X_train.shape[1]
forest_third = RandomForestRegressor(n_estimators=500, max_features=(n_features //
3), random_state=RANDOM_SEED)
forest_third.fit(X_train, y_train)
forest_third_mse = mean_squared_error(y_test, forest_third.predict(X_test))

forest_sqrt = RandomForestRegressor(n_estimators=500, max_features="sqrt",
random_state=RANDOM_SEED)
forest_sqrt.fit(X_train, y_train)
forest_sqrt_mse = mean_squared_error(y_test, forest_sqrt.predict(X_test))

print(f"Random forest (max_features={n_features // 3}, ~p/3) test MSE:
{forest_third_mse:.4f}")
print(f"Random forest (max_features='sqrt') test MSE: {forest_sqrt_mse:.4f}")

sqrt_importances = pd.Series(
    forest_sqrt.feature_importances_, index=X_train.columns
).sort_values(ascending=False)

third_importances = pd.Series(
    forest_third.feature_importances_, index=X_train.columns
).sort_values(ascending=False)

print("\nTop 10 feature importances (max_features='sqrt' forest):")
print(sqrt_importances.head(10))

print(f"\nTop 10 feature importances (max_features={n_features // 3}, ~p/3
forest):")
print(third_importances.head(10))
```

```
Random forest (max_features=13, ~p/3) test MSE: 151.8414
Random forest (max_features='sqrt') test MSE: 149.9331

Top 10 feature importances (max_features='sqrt' forest):
TuitionOut     0.095817
AvgSAT         0.089384
Pell           0.082150
TuitionIn      0.065945
Cost           0.063520
InstructFTE    0.059939
MidACT         0.054960
FacSalary      0.053342
Black          0.042175
TuitionFTE     0.041658
dtype: float64

Top 10 feature importances (max_features=13, ~p/3 forest):
AvgSAT         0.143370
TuitionOut     0.118075
Pell           0.097318
MidACT         0.058893
TuitionIn      0.055561
Cost           0.054943
InstructFTE    0.050017
FacSalary      0.040939
Black          0.039888
AdmitRate      0.036336
dtype: float64
```

(e) From the forest in (d): which predictors look most important?

Also name one predictor that is important in the forest but barely appears in the single tree from (b)–(c), and give a one-sentence reason that can happen.

The two random forests performed basically the same. The sqrt model did slightly better, but the difference in test MSE was only about 1.9. Both models identified AvgSAT, TuitionOut, and Pell as the most important predictors. The p/3 model relied more heavily on AvgSAT, while the sqrt model spread importance more evenly across TuitionIn, Cost, InstructFTE, FacSalary, and other test-score variables. TuitionIn is the 4th most important and is not present in the single trees in previous problems. The reason is likely that TuitionIn and TuitionOut are highly correlated and compete in a single tree; TuitionOut is slightly better overall apparently. With random forests, in some trees, TuitionOut does NOT appear in the optional features and so TuitionIn is chosen. The random forrest is the accumulation/average of all of these trees.

(f) Report the test MSE for the forest and compare it with the pruned tree. Also report test MSE by Control (Public / Private / Profit) for both models. What do you observe?

In [9]:
```
#  better of the two forests from (d)
forest = forest_sqrt if forest_sqrt_mse <= forest_third_mse else forest_third
```

```
pred_tree = pruned_tree.predict(X_test)
pred_forest = forest.predict(X_test)

print(f"Pruned tree test MSE: {mean_squared_error(y_test, pred_tree):.2f}")
print(f"Random forest test MSE: {mean_squared_error(y_test, pred_forest):.2f}\n")

# MSE by Control
rows = []
for level in ["Public", "Private", "Profit"]:
    mask = (control_test == level).values
    rows.append({
        "Control": level,
        "n_test": mask.sum(),
        "Pruned tree MSE": mean_squared_error(y_test[mask], pred_tree[mask]),
        "Forest MSE": mean_squared_error(y_test[mask], pred_forest[mask]),
    })

mse_by_control = pd.DataFrame(rows).set_index("Control").round(2)
display(mse_by_control)
```

```
Pruned tree test MSE: 210.12
Random forest test MSE: 149.93
```

Control	n_test	Pruned tree MSE	Forest MSE
Public	173	165.38	64.28
Private	335	202.35	171.81
Profit	40	468.79	337.20

The random forest has lower test MSE than the pruned tree for every Control type. That checks out given we're averaging many trees together to arrive at the most predictive model. The fact that Private school MSE is largest and Public MSE is smallest is probably a symptom of how noisy the data is.

(g) Fit a GBM. Choose at least two learning rates (shrinkage). In R use distribution="gaussian" in gbm.

Choose the number of trees by cross-validation on the training data. Evaluate that choice once on the test set. You may plot test MSE versus number of trees for the two learning rates as a diagnostic of over/underfitting; do not use that plot to pick the number of trees.

Interpret the two most influential variables.

In [10]:
```
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.model_selection import GridSearchCV

params_slow_learning_rate = {
    "max_depth": 4,
    "min_samples_split": 5,
```

```
    "learning_rate": 0.01,
    "loss": "squared_error",
    "random_state": 598
}
params_fast_learning_rate = {
    "max_depth": 4,
    "min_samples_split": 5,
    "learning_rate": 0.1,
    "loss": "squared_error",
    "random_state": 598
}

gbr_slow = GradientBoostingRegressor(**params_slow_learning_rate)
gbr_fast = GradientBoostingRegressor(**params_fast_learning_rate)

param_grid_fast = {"n_estimators": list(range(25, 501, 25))}
param_grid_slow = {"n_estimators": list(range(250, 3001, 250))}

cv_slow = GridSearchCV(gbr_slow, param_grid_slow,
scoring="neg_mean_squared_error", cv=5)
cv_fast = GridSearchCV(gbr_fast, param_grid_fast,
scoring="neg_mean_squared_error", cv=5)

cv_slow.fit(X_train, y_train)
cv_fast.fit(X_train, y_train)

print(f"Slow (0.01): trees={cv_slow.best_params_['n_estimators']}, CV MSE={-
cv_slow.best_score_:.2f}")
print(f"Fast (0.1): trees={cv_fast.best_params_['n_estimators']}, CV MSE={-
cv_fast.best_score_:.2f}")

if -cv_slow.best_score_ <= -cv_fast.best_score_:
    gbm = cv_slow.best_estimator_
else:
    gbm = cv_fast.best_estimator_

gbm_mse = mean_squared_error(y_test, gbm.predict(X_test))

print(f"\nChosen GBM: learning rate={gbm.learning_rate}, trees={gbm.n_estimators},
test MSE={gbm_mse:.2f}")

gbm_importances = pd.Series(gbm.feature_importances_,
index=X_train.columns).sort_values(ascending=False)
print("\nTop 10 feature importances (chosen GBM):")
print(gbm_importances.head(10))
```

```
Slow (0.01): trees=750, CV MSE=169.44
Fast (0.1): trees=75, CV MSE=168.86

Chosen GBM: learning rate=0.1, trees=75, test MSE=162.41

Top 10 feature importances (chosen GBM):
AvgSAT         0.275300
Pell           0.137326
TuitionOut     0.108582
InstructFTE    0.053164
AdmitRate      0.038172
FacSalary      0.037489
Black          0.032380
Enrollment     0.031913
Cost           0.031679
PartTime       0.030404
dtype: float64
```

The two most important features, according to GBM, are AvgSAT and Pell

AvgSAT measures school selectivity with respect to SAT scores. This selectivity correlates highly with a high completion rate. There are likely a variety of reasons that this is the case. A simple one: having students with a higher SAT score means more of them are likely to complete degrees. What we can interpret is that the model shows that this is an important feature, and we agree that this is an important feature. Pell measures the percentage of undergraduate students with Pell grants. Pell grants are typically awarded to low-income students. Low-income students may have financial struggles which affect their ability to complete school. Again, this is just an idea. What the data supports for both of these features is that they are highly associated with completion rate.

(h) Fit one linear baseline on encoded features (ordinary least squares or ridge is enough).

Encode categoricals and standardize numerics using the training set only.

Compare test MSE of: mean baseline, Control-mean baseline, pruned tree, forest, GBM, and the linear model. Comment.

In [11]:
```
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import StandardScaler

# standardize numerics using training set only
scale_cols = num_cols.drop("ID")
scaler = StandardScaler()
X_train_scaled = X_train.copy()
X_test_scaled = X_test.copy()
X_train_scaled[scale_cols] = scaler.fit_transform(X_train[scale_cols])
X_test_scaled[scale_cols] = scaler.transform(X_test[scale_cols])

linear = LinearRegression()
linear.fit(X_train_scaled, y_train)
pred_linear = linear.predict(X_test_scaled)
```

```
mean_mse = mean_squared_error(y_test, np.full(len(y_test), y_train.mean()))
control_means = y_train.groupby(control_train).mean()
control_mean_mse = mean_squared_error(y_test, control_test.map(control_means))
linear_mse = mean_squared_error(y_test, pred_linear)

print("Mean baseline test MSE:", mean_mse)
print("Control-mean baseline test MSE:", control_mean_mse)
print("Pruned tree test MSE:", pruned_mse)
print("Random forest test MSE:", mean_squared_error(y_test, pred_forest))
print("GBM test MSE:", gbm_mse)
print("Linear regression test MSE:", linear_mse)
```

```
Mean baseline test MSE: 405.409540640721
Control-mean baseline test MSE: 397.9536747099963
Pruned tree test MSE: 210.12377668826724
Random forest test MSE: 149.933124189708
GBM test MSE: 162.40910147458598
Linear regression test MSE: 168.95721458094994
```

The mean baseline is simply the mean of response variable. Sets baseline for other models. Control-mean baseline includes one predictor, public/private/profit. Shows slight improvement from baseline. Pruned tree comes from Part 1c. We found that it was the best single decision tree based on a 5-fold cross-validation on the training data. It is the lowest performing tree model, and performs worse than a linear regression here. Random forest comes from Part 1d. The prediction comes from the average output of 500 regression trees fit to the training data. We can see that the forest performs better than the pruned tree. Gradient Boosted Machine (GBM) was fitted in Part 1g. It builds one tree after another, each fitted to the errors of the trees before it. We see a better MSE than the pruned tree, but a worse MSE than the random forest. It also takes longer to train, so it's hard to see the benefit. Here we used a 5-fold CV over 12 tree counts at rate .01 and 20 at rate .1, about 124,000 trees in total. Linear regression is being tested in the above cell, Part 1h. The advantage of a linear regression is its simplicity, and it does perform better than the pruned tree, but we can see that the more complex tree models (GBM and random forest) perform better for this data set.

Part II: Eight schools

From your test set only, choose eight schools and print them: 3 Public, 3 Private, and 2 Profit. Do not pick the eight schools by looking at the residuals or by observed completion. Training-set rows are not allowed.

Then create the following table: Name, Control, Pell, predicted CompRate from your preferred Part I model, observed CompRate, residual (observed minus predicted).

In [12]:
```
test_public_schools = X_test[control_test == "Public"].sample(n=3,
random_state=RANDOM_SEED)
```

```
test_private_schools = X_test[control_test == "Private"].sample(n=3,
random_state=RANDOM_SEED)
test_profit_schools = X_test[control_test == "Profit"].sample(n=2,
random_state=RANDOM_SEED)

chosen_schools = pd.concat([test_public_schools, test_private_schools,
test_profit_schools])

name = X.loc[chosen_schools.index]["Name"]
control = ["Public"] * 3 + ["Private"] * 3 + ["Profit"] * 2
pell = chosen_schools['Pell']
pred_comp_rate = forest.predict(chosen_schools)
observed_comp_rate = y_test.loc[chosen_schools.index]
residual = observed_comp_rate - pred_comp_rate
midact_missing = chosen_schools['MidACTMissing']
avgsat_missing = chosen_schools['AvgSATMissing']

table = pd.DataFrame({
    "Name": name,
     "Control": control,
     "Pell": pell,
     "Predicted CompRate": pred_comp_rate,
     "Observed CompRate": observed_comp_rate,
     "Residual": residual,
     "Missing MidACT": midact_missing,
     "Missing AvgSAT": avgsat_missing

})

table
```

Out[12]:

	Name	Control	Pell	Predicted CompRate	Observed CompRate	Residual	Missing MidACT	Missing AvgSAT
1637	University of Wisconsin-Madison	Public	14.9	83.3332	89.2	5.8668	False	False
1283	Pennsylvania State University-Penn State Shenango	Public	35.4	39.3988	23.3	-16.0988	True	True
352	Illinois State University	Public	30.3	57.4474	67.0	9.5526	False	False
853	Caldwell University	Private	47.1	56.3790	68.0	11.6210	True	False
1978	Congregation Talmidei Mesivta Tiferes Shmiel A...	Private	72.7	36.8780	14.0	-22.8780	True	True
661	Adrian College	Private	30.9	54.0236	50.6	-3.4236	False	False
1435	The Art Institute of Houston	Profit	48.9	25.1874	40.8	15.6126	True	True
1855	American Medical Sciences Center	Profit	63.1	49.8056	68.4	18.5944	True	True

Based on your results and the models from Part I, answer the following questions (one or two sentences is sufficient):

(1) Which of the eight schools has the largest absolute residual? Did the model over- or under-predict, and what kind of school is it (using Control and the name)?

The random forest was chosen as the preferred Part I model. Congregation Talmidei Mesivta Tiferes Shmiel Aleksander has the largest absolute residual of 22.8780. The model over-predicted for this observation since the observed completion rate was 14.0% and the prediction completion rate was 36.8780%. The school is a private school.

(2) Do the errors in your table line up with Control, with missing test scores, or with neither?

The errors in the table line up with the missing test scores as the the three chosen schools that do not have missing test scores have the lowest errors.

(3) These eight numbers are predictions, not a ranking of school quality and not an effect of changing Pell, spending, or admissions. In one sentence, state a conclusion someone might draw from your table that the model does not support.

Someone might see that schools with a higher Pell share tend to have lower predicted completion rates and conclude that lowering a school's Pell share, or enrolling fewer low-income students, would raise its completion rate. The model does not support this because it only finds associations in observed data. Pell share is correlated with other factors such as selectivity, resources, and Control, so the table says nothing about what would happen if a school changed its admissions or aid, and a low completion rate is not evidence of poor school quality.